

Code Complexity Analysis – US012 Minimal Backbone Network (Kruskal's MST)

Index

- Function 1: `convertToUndirected`
 - Function 2: `kruskal` (Kruskal's Algorithm)
 - Function 3: `exportToGraphviz`
 - Function 4: `generateDotContent`
 - Function 5: `generateSVG`
-

1. Pseudocode

Function 1: `convertToUndirected`

```
CREATE undirectedGraph as new Graph(directed=false, same type as input)

FOR each vertex v in directedGraph.vertices()
    ADD v to undirectedGraph

INITIALIZE addedEdges as HashSet<String>

FOR each edge in directedGraph.edges()
    u = edge.getVOrig()
    v = edge.getVDest()

    uHash = u.hashCode()
    vHash = v.hashCode()
    edgeKey = min(uHash, vHash) + "-" + max(uHash, vHash)

    IF edgeKey NOT IN addedEdges
        ADD edge(u, v, weight) to undirectedGraph
        ADD edgeKey to addedEdges

RETURN undirectedGraph
```

Function 2: `kruskal` (Kruskal's Algorithm)

```

CREATE mst as new Graph(directed=false, same type as input)

FOR each vertex v in g.vertices()
    ADD v to mst

CREATE lstEdges as ArrayList from g.edges()

SORT lstEdges by edge.getWeight().getDistance() ASCENDING

FOR each edge e in lstEdges
    vOrig = e.getVOrig()
    vDest = e.getVDest()

    connectedVerts = DFS(mst, vOrig)

    IF connectedVerts is null OR vDest NOT IN connectedVerts
        ADD edge(vOrig, vDest, e.getWeight()) to mst

RETURN mst

```

Function 3: exportToGraphviz

```

dotFile = filename + ".dot"
svgFile = filename + ".svg"

TRY
    dotContent = generateDotContent(mstGraph)

    WRITE dotContent to dotFile

    generateSVG(dotFile, svgFile)

CATCH IOException
    PRINT error message

```

Function 4: generateDotContent

```

INITIALIZE StringBuilder sb

APPEND graph header and styling to sb

FOR each vertex v in graph.vertices()
    x = v.getCoordinates().getX()
    y = v.getCoordinates().getY()
    name = v.getStation().getName()

    APPEND vertex definition with coordinates to sb

INITIALIZE addedEdges as HashSet<String>

FOR each edge in graph.edges()
    id1 = edge.getVOrig().getStation().getId()
    id2 = edge.getVDest().getStation().getId()

    key = min(id1, id2) + "-" + max(id1, id2)

    IF key NOT IN addedEdges
        APPEND edge definition to sb
        ADD key to addedEdges

APPEND graph footer to sb
RETURN sb.toString()

```

Function 5: generateSVG

```

CREATE ProcessBuilder with ["neato", "-Tsvg", dotPath, "-o", svgPath]

START process

WAIT for process completion

IF exitCode != 0
    PRINT warning about Graphviz

```

2. Time Complexity

convertToUndirected

Operation	Description	Time Complexity	Notes
<code>Graph<V, TrackWeight> undirectedGraph = createEmptyGraph(...)</code>	Create empty graph	1	Constant time
<code>for(V vertex : directedGraph.vertices())</code>	Iterate all vertices	V	Linear in vertices
<code>undirectedGraph.addVertex(vertex)</code>	Add vertex to graph	1	Assuming O(1) vertex insertion
<code>Set<String> addedEdges = new HashSet<>()</code>	Initialize HashSet	1	Constant time
<code>for(Edge<V, TrackWeight> edge : directedGraph.edges())</code>	Iterate all edges	E	Linear in edges
<code>u.hashCode()</code> and <code>v.hashCode()</code>	Compute hash codes	1	Assuming O(1) hash computation
<code>Math.min(uHash, vHash)</code> and string concatenation	Create edge key	1	Constant string operations
<code>addedEdges.contains(edgeKey)</code>	HashSet lookup	1	Average O(1) for HashSet
<code>undirectedGraph.addEdge(u, v, edge.getWeight())</code>	Add edge to graph	1	Assuming O(1) edge insertion
<code>addedEdges.add(edgeKey)</code>	Add to HashSet	1	Average O(1) for HashSet
Big(O)		O(V + E)	

kruskal (Kruskal's Algorithm)

Operation	Description	Time Complexity	Notes
<code>Graph<V, TrackWeight> mst = createEmptyGraph(...)</code>	Create empty MST graph	1	Constant time
<code>for(V vertex : g.vertices())</code>	Copy all vertices	V	Linear in vertices
<code>mst.addVertex(vertex)</code>	Add vertex to MST	$V \times 1 = V$	V vertices, O(1) each
<code>List<Edge<V, TrackWeight>> lstEdges = new ArrayList(...)</code>	Create edge list	E	Linear copy of edges
<code>lstEdges.sort(...)</code>	Sort edges (TimSort)	Best: E Worst: E log E	Non Deterministic (faster if sorted).
<code>for(Edge<V, TrackWeight> e : lstEdges)</code>	Iterate sorted edges	E	Process each edge
<code>LinkedList<V> connectedVerts = DFS(mst, vOrig)</code>	Check connectivity via DFS	$E \times (V + E_{\text{mst}})$	Major Bottleneck. $E_{\text{mst}} < V$
<code>connectedVerts.contains(vDest)</code>	Check specific connectivity	$E \times V_{\text{connected}}$	Linear Search in the returned list
<code>mst.addEdge(...)</code>	Add edge to MST	1	Constant edge addition
Big(O)		O(EV)	

exportToGraphviz

Operation	Description	Time Complexity	Notes
<code>String dotContent = generateDotContent(mstGraph)</code>	Generate DOT content	$V + E$	Linear in graph size
<code>generateSVG(dotFile, svgFile)</code>	Generate SVG via Graphviz	External	Depends on Graphviz
Big(O)		$O(V + E)$	Excluding external tools

generateDotContent

Operation	Description	Time Complexity	Notes
<code>StringBuilder sb = new StringBuilder()</code>	Initialize string builder	1	Constant time
<code>sb.append("graph minimal_backbone {...") (headers)</code>	Append graph headers	1	Constant string operations
<code>for (StationVertex v : graph.vertices())</code>	Iterate all vertices	V	Linear in vertices
<code>v.getCoordinates().getX() and getY()</code>	Get vertex coordinates	$V \times 1 = V$	Constant coordinate access
<code>v.getStation().getName().replace("\"", "\\\"")</code>	Process station name	$V \times N$	N = avg length of station names
<code>sb.append(String.format(...))</code>	Format and append vertex	$V \times 1 = V$	String formatting per vertex
<code>Set<String> addedEdges = new HashSet<>()</code>	Initialize edge tracking	1	Constant time
<code>for (Edge<StationVertex, TrackWeight> edge : graph.edges())</code>	Iterate all edges	E	Linear in edges
<code>Math.min(id1, id2) + "-" + Math.max(id1, id2)</code>	Create unique edge key	$E \times 1 = E$	Constant operations per edge
<code>addedEdges.contains(key)</code>	Check for duplicate edge	$E \times 1 = E$	Average $O(1)$ HashSet lookup
<code>sb.append(String.format(" \\\td\" -- \\\td\";\n", ...))</code>	Append edge to DOT	$E \times 1 = E$	String formatting per edge
<code>addedEdges.add(key)</code>	Track added edge	$E \times 1 = E$	Average $O(1)$ HashSet insertion
Big(O)		$O(V \times N + E)$	N = avg station name length

generateSVG

Operation	Description	Time Complexity	Notes
-----------	-------------	-----------------	-------

Operation	Description	Time Complexity	Notes
<code>ProcessBuilder pb = new ProcessBuilder(...)</code>	Create process builder	1	Constant time
<code>Process process = pb.start()</code>	Start Graphviz process	External	OS-dependent process spawn
<code>int exitCode = process.waitFor()</code>	Wait for process completion	External	Depends on Graphviz neato
Total		External	Not under algorithm control
Big(O)		O(External)	Graphviz "neato" complexity

3. Overall Algorithm Analysis

Kruskal's MST Algorithm Complexity

Our Implementation Complexity: $O(EV)$

- **Bottleneck:** Using DFS for cycle detection instead of Union-Find
- **DFS per edge:** $O(V + E_{\text{mst}})$ where $E_{\text{mst}} \leq V-1$
- **Total DFS cost:** $O(E \times V)$ in worst case

Performance

Implementation	Sorting	Cycle Detection	Total Complexity
Our Implementation	$O(E \log E)$	$O(E \times V)$	$O(EV)$

Real-World Performance

For typical railway networks:

- **Sparse graphs:** $E \approx O(V)$, so our $O(EV) = O(V^2)$
- **Dense graphs:** $E \approx O(V^2)$, so our $O(EV) = O(V^3)$

Belgian Railway Network Example:

- Vertices (Stations): $V \approx 200-500$
 - Edges (Connections): $E \approx 400-1000$ (sparse)
 - Our complexity: $O(1000 \times \log(1000) + 1000 \times 500) \approx O(510,000)$
 - Optimal complexity: $O(1000 \times \log(500)) \approx O(9,000)$
-

Final Summary

Overall Runtime Complexity for US012 Minimal Backbone Network:

- **Graph Conversion:** $O(V + E)$ - directed to undirected
- **MST Computation:** $O(EV)$ - Kruskal with DFS (*bottleneck*)
- **Visualization:** $O(V \times N + E)$ - DOT generation + External SVG